Heart Disease Prediction MLOps

End-to-end development, experiment tracking, serving, and operational readiness

Machine Learning Operations (MLOps) | Assignment 01

Dataset: UCI Heart Disease, Cleveland subset

Repository: github.com/adityathiyyagura/mlops-heart-disease-assignment

Executive summary

This project develops a reproducible binary classifier and wraps it in an API-oriented MLOps workflow. It covers source data acquisition, cleaning, exploratory analysis, model comparison, MLflow logging, automated tests, Docker packaging, Kubernetes configuration, request logging, and Prometheus metrics.

The evaluated models are Logistic Regression and Random Forest. Model selection uses mean ROC-AUC from five-fold stratified cross-validation on development data. A separate stratified 20% holdout is kept out of cross-validation and reported as a final test set.

Selected by CV: Random Forest (mean CV ROC-AUC 0.888). Held-out test ROC-AUC: 0.940. These results are educational model metrics, not a clinical risk assessment or validation.

System at a glance

Data source -> cleaning/EDA -> stratified CV and held-out test
MLflow tracking -> selected fitted pipeline -> FastAPI
Docker image -> Kubernetes manifest | /metrics -> Prometheus scrape

Prepared from the project implementation and locally verified outputs. Deployment evidence is identified explicitly where it has not yet been captured.

1. Dataset acquisition and preparation

Data source and scope

The project uses the processed Cleveland Heart Disease dataset from the UCI Machine Learning Repository. It has 13 predictor columns and one disease target. The original target has values 0 through 4; this project maps 0 to no disease and any positive value to disease present.

The source is obtained by the download utility at scripts/download_data.py. The raw file is retained at data/heart.csv, while data/heart_cleaned.csv contains the complete-case data used for the deliverable. The cleaning path converts '?' to missing values and numeric-coerces feature columns.

Dataset counts

Measure	Observed value	Interpretation
Rows loaded for EDA	303	Target-valid source records before feature-row removal
Rows in cleaned CSV	297	Six records removed because feature values were missing
Columns	14	13 predictors plus binary target
Clean target 0	160	No disease in cleaned records
Clean target 1	137	Disease present in cleaned records
Missing values after cleaning	0	Verified in data/heart_cleaned.csv

Cleaning and leakage controls

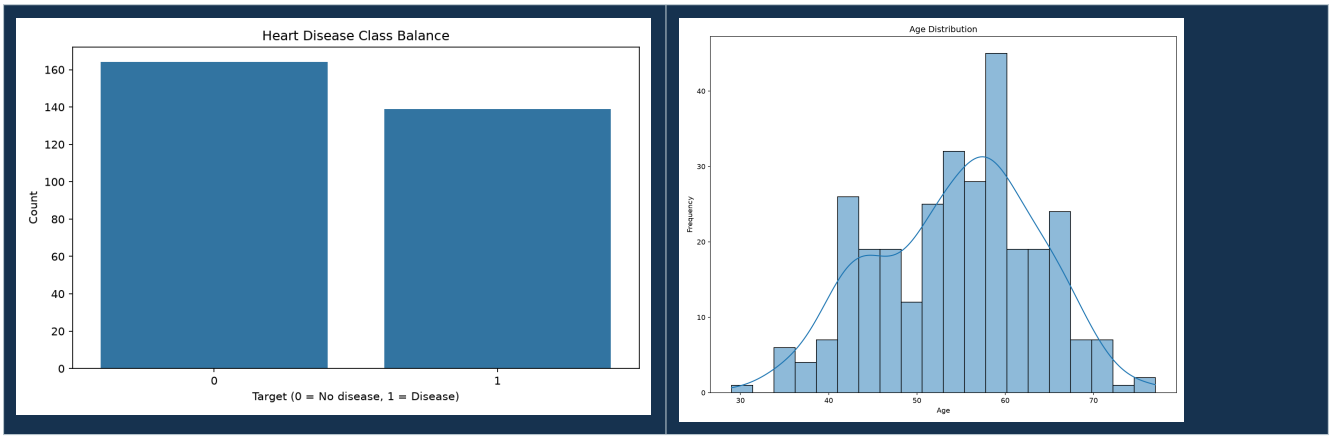
- The target is excluded from the input matrix and converted to a binary integer label.
- Rows with missing predictor values are removed before training; no test labels are used to fit the estimator.
- The train/test split is stratified to retain approximate class proportions in both partitions.
- Scaling is inside the Logistic Regression pipeline, so it is fitted independently within each CV fold and on development data for the final model.

This is complete-case handling rather than imputation. It is simple and reproducible for this small dataset, but may introduce bias if missingness is systematic.

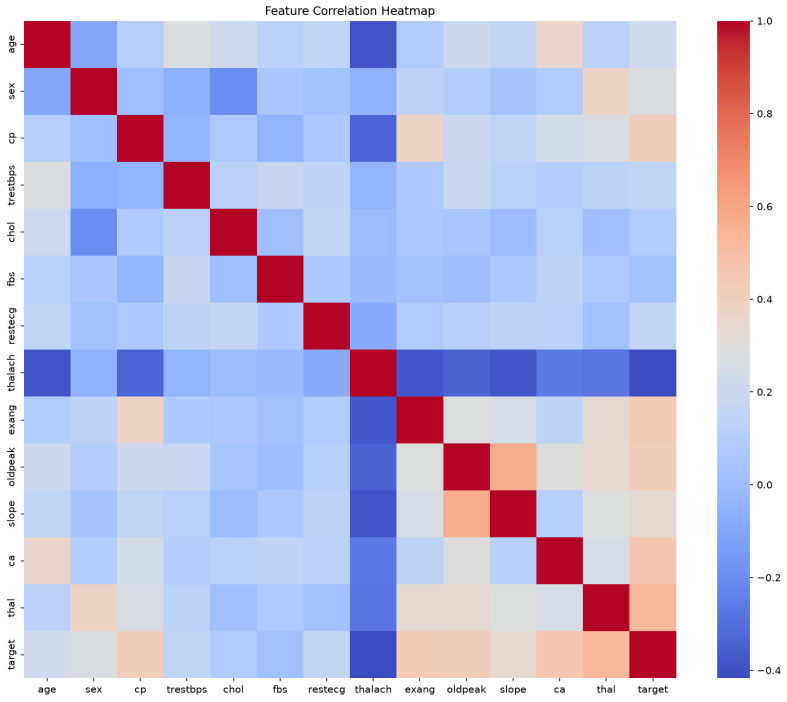
2. Exploratory data analysis

EDA is generated from the target-valid source data before feature rows with missing values are excluded. This is why EDA class counts (164 / 139) differ from cleaned modeling counts (160 / 137). The notebook and scripts save plots as artifacts for review.

Target distribution and age



Correlation overview



Interpretation

The target classes are moderately imbalanced rather than severely skewed. Age spans adult and older patients, with most records concentrated in middle-to-late adulthood. The correlation heatmap provides a first-pass view of linear associations; it is descriptive and does not establish causality or feature importance. Missing values in source predictors are handled after this descriptive stage for model preparation.

3. Features and model pipeline

The model uses the dataset's 13 input attributes. Several fields are integer-coded categories in the source data; the current implementation treats them as numeric values. Logistic Regression uses standardization. Random Forest uses tree splits and does not require scaling.

Input schema

Feature group	Fields	Preparation
Demographic	age, sex	Numeric; source coding retained
Clinical measurements	trestbps, chol, thalach, oldpeak	Numeric coercion; complete-case rows
Clinical categories	cp, fbs, restecg, exang, slope, ca, thal	Integer codes retained as numeric
Target	target	0 = absent; 1-4 mapped to present

Estimator construction

Candidate	Pipeline	Reproducibility controls
Logistic Regression	StandardScaler -> LogisticRegression(max_iter=1000)	random_state=42
Random Forest	RandomForestClassifier(n_estimators=200, class_weight=balanced)	random_state=42

Inference uses the same named feature order as training. The API validates that all expected fields are present through its request schema. Model output is a binary class and the maximum class probability is returned as confidence; this confidence is not calibrated clinical probability.

Potential next improvement: one-hot encode nominal categories and use a ColumnTransformer. That would avoid implying an ordinal distance between category codes where the clinical coding does not support one.

4. Validation protocol and model results

A stratified 80/20 split is made once using random_state=42. Five-fold StratifiedKFold (shuffle enabled, random_state=42) runs on the 80% development data. Each fold preserves class representation. Model selection uses the mean CV ROC-AUC; the 20% holdout is reserved for a separate final comparison.

Cross-validation on development partition (mean +/- sample standard deviation)

Model	Accuracy	Precision	Recall	ROC-AUC
Logistic Regression	0.827 +/- 0.066	0.849 +/- 0.096	0.769 +/- 0.108	0.869 +/- 0.077
Random Forest	0.810 +/- 0.084	0.833 +/- 0.124	0.760 +/- 0.142	0.888 +/- 0.062

Final held-out test metrics

Model	Accuracy	Precision	Recall	ROC-AUC
Logistic Regression	0.833	0.846	0.786	0.950
Random Forest (CV-selected)	0.867	0.885	0.821	0.940

Selection and interpretation

Random Forest has the higher mean CV ROC-AUC (0.888 versus 0.869), so it is selected by the predeclared selection metric. On the single held-out split it also has higher accuracy, precision, and recall; Logistic Regression has a slightly higher held-out ROC-AUC. This divergence illustrates why cross-validation and a separately held-out result should both be reported rather than selecting on the test set.

The standard deviations indicate fold-to-fold variability on a small dataset. These point estimates have substantial uncertainty and are not claims of generalization to a different hospital, population, or contemporary cohort.

5. Experiment tracking and reproducibility

MLflow is used to create one run per model. Each run records estimator parameters, a serialized sklearn model, held-out metrics, and CV summary metrics. CV metric names follow the form cv__mean and cv__std; holdout values are stored separately with a held_out_ prefix.

Tracked CV summaries

Metric	Logistic Regression mean / std	Random Forest mean / std
Accuracy	0.827 / 0.066	0.810 / 0.084
Precision	0.849 / 0.096	0.833 / 0.124
Recall	0.769 / 0.108	0.760 / 0.142
ROC-AUC	0.869 / 0.077	0.888 / 0.062

Project artifacts and entry points

Path	Purpose
heart_disease_mlop/data.py	Download, load, and prepare dataset
heart_disease_mlop/modeling.py	Pipelines, CV, holdout evaluation, MLflow, model persistence
heart_disease_mlop/train.py	Command-line model training entry point
notebooks/	EDA and model comparison notebooks
artifacts/heart_disease_model.joblib	Persisted selected fitted pipeline
mlruns/	Local MLflow tracking data (runtime output)

A fixed random seed is used for splitting, CV shuffling, and estimators. For stronger production reproducibility, pin all dependency versions and record a dataset checksum; dependency ranges are currently specified in requirements.txt.

6. Prediction API and container

The FastAPI service exposes health and prediction routes. A request contains the 13 clinical predictor values; the response contains a binary prediction, a human-readable risk label, and a confidence score. The request schema rejects missing or malformed fields before model inference.

API surface

Route	Method	Purpose
/health	GET	Basic process health check
/predict	POST	Validate features and return class, label, confidence
/metrics	GET	Expose Prometheus text-format request counters and latency

Example request

```
{
  "age": 52, "sex": 1, "cp": 0, "trestbps": 125,
  "chol": 212, "fbs": 0, "restecg": 1, "thalach": 168,
  "exang": 0, "oldpeak": 1.0, "slope": 2, "ca": 2, "thal": 3
}
```

Local run commands

```
python -m heart_disease_mlop.train
docker build -t heart-disease-api .
docker run --rm -p 8000:8000 heart-disease-api
curl http://127.0.0.1:8000/health
```

The model is saved as a joblib pipeline and loaded by the service. The Dockerfile installs requirements, copies the project, exposes port 8000, and starts Uvicorn. A local TestClient prediction request returned HTTP 200 during verification; the confidence value is a model score, not a calibrated medical probability.

The API is an educational demonstration and must not be used to diagnose or make treatment decisions.

7. Testing and continuous integration

Automated tests cover the main data interface, model construction/evaluation, and API prediction/metrics routes. The latest local verification ran six tests successfully. The project workflow is triggered on pushes and pull requests to main/master.

Current workflow stages

Stage	Action	Status in workflow
Checkout	Fetch repository source	Configured
Runtime	Set up Python 3.12	Configured
Dependencies	Install requirements.txt	Configured
Tests	Run pytest	Configured
Training	Run python -m heart_disease_mlop.train	Configured

Quality gates and evidence

- Local test result: 6 passed. Warnings include FastAPI lifecycle and TestClient deprecations; these do not fail the suite.
- Training was executed locally and produced the CV-selected Random Forest artifact and MLflow runs.
- The current workflow does not yet include a lint step or upload generated model/report artifacts as workflow artifacts.
- A successful hosted GitHub Actions run should be checked and captured from the repository Actions page before final submission.

For a stronger CI gate, add Ruff (or another selected linter), upload the trained model and test logs with actions/upload-artifact, and ensure training produces artifacts in a clean runner. Avoid committing transient MLflow state as a substitute for workflow artifacts.

8. Deployment and observability

Deployment configuration

The repository includes a Kubernetes Deployment with two replicas and a LoadBalancer Service on port 80 targeting the API container on port 8000. The manifest is a deployment configuration, not proof of a completed cluster rollout. No public cloud endpoint is claimed in this report.

For local verification, build the Docker image, configure a local Kubernetes context (for example Docker Desktop or Minikube), apply k8s/deployment.yaml, and inspect pods/service status. If the local cluster cannot expose a LoadBalancer, use kubectl port-forward and document the resulting local URL and access steps.

Request monitoring

Signal	Implementation	Use
Request count	http_requests_total labels method/path/status	Traffic and response status trends
Latency	http_request_duration_seconds histogram	Request duration distribution
Application log	Method, path, status, elapsed seconds	Local/container request diagnostics
Scrape config	monitoring/prometheus.yml	Prometheus target configuration

Observed locally

The API smoke test returned HTTP 200 for /predict and /metrics, and request log entries were emitted. The metrics endpoint included http_requests_total. Prometheus scraping is configured, but a running Prometheus/Grafana dashboard and real deployment screenshot remain deployment evidence to capture separately.

Before claiming deployment completion, record the cluster context, image tag, rollout status, service address or port-forward command, successful health/prediction requests, and timestamped screenshots.

9. Limitations, conclusion, and references

Limitations and responsible use

- Small, historical Cleveland subset: performance estimates may not transfer to another clinical setting or current population.
- Complete-case row removal can bias estimates if missingness is not random.
- Integer-coded categories are treated as numeric; future work should review appropriate categorical encoding with domain expertise.
- The reported confidence is not probability calibration, and the model is not a diagnostic device.
- The held-out test set is one split; external validation and calibration analysis are not performed.

Conclusion

The implementation demonstrates a reproducible progression from public data to a tracked classifier, tested API, container definition, deployment manifest, and request metrics. Five-fold stratified CV selects Random Forest by mean ROC-AUC while the final holdout is kept separate. The remaining submission evidence is operational: confirm a hosted Actions run, execute the chosen local/cloud deployment, and capture genuine screenshots and a live walkthrough.

References

- UCI Machine Learning Repository, Heart Disease dataset: https://archive.ics.uci.edu/dataset/45/heart+disease
- Project repository: https://github.com/adityathiyyagura/mlops-heart-disease-assignment
- scikit-learn model evaluation and StratifiedKFold documentation: https://scikit-learn.org/stable/modules/cross_validation.html
- MLflow Tracking documentation: https://mlflow.org/docs/latest/ml/tracking/
- FastAPI documentation: https://fastapi.tiangolo.com/

Submission evidence checklist

Code, data, notebooks/scripts, tests, Dockerfile, requirements, CI workflow, report, and Kubernetes manifest are included in the repository. Attach genuine hosted workflow/deployment screenshots and replace the generated slideshow with an actual pipeline recording before final submission.